
Universidad Aeronáutica de Querétaro

Embedded Systems Laboratory

SCADE One Laboratory Series

Practice #4

Talking to the World: A Framed UART Telemetry Link

Elaborated by: Leonardo Franco García

Student ID: 12148

Date: July 22, 2026

Full name: _____

Student ID: _____

Date: _____

1 Learning Objectives

Learning Objectives

By the end of this practice, the student will be able to:

- ✓ **Separate model logic from communication:** keep the SCADE-generated code untouched and confine every byte of protocol handling to the glue layer, reinforcing the boundary established in Practice 3.
- ✓ **Configure and drive a UART peripheral:** initialize `UART_NUM_2` on the ESP32 with an explicit 8N1, `esp_driver_uart` API.
- ✓ **Design a binary telemetry frame:** define a fixed-length frame (header, payload, checksum) that carries the model state, and justify each field in terms of framing and error detection.
- ✓ **Receive and validate the frame:** read the frame back through an internal loopback, verify the header and checksum, and reject corrupted frames — proving the link end to end without any external device.

2 Introduction

A landing gear controller is never alone: it must report its state to a cockpit display, a maintenance bus, or a flight-data recorder, and every one of those links is a serial protocol. This practice builds the first and most fundamental of them — an asynchronous UART link — while keeping the SCADE model exactly as generated in Practice 3. The traffic-light state is packed into a compact binary frame, transmitted on a dedicated UART port, and read back through a single loopback wire so the entire transmit-and-verify path can be exercised with no hardware beyond the board already in hand. UART is the foundation on which the SPI, I2C, and CAN links surveyed here are later built.

3 Bill of Materials

#	Component / Software	Qty	Notes
1	SCADE One (Ansys)	1	Swan code generation enabled
2	ESP32 DevKit V1	1	Environment from Practice 2/3
3	USB-A to Micro-USB cable	1	Data cable (console UART0)
4	ESP-IDF (v5.x)	1	Installed in Practice 2
5	Generated <code>scade_gen</code> component	1	Reused from Practice 3 (<code>TrafficLight</code>)
6	Jumper wire (M-M)	1	UART2 loopback: GPIO 4 to GPIO 5
7	Three LEDs + 220Ω resistors	—	Optional, carried over from Practice 3

Wiring

The only new connection is the **loopback jumper**: a single wire from **GPIO 4** to **GPIO 5**. UART2 is remapped to these two free general-purpose pins with `uart_set_pin`, so its TX drives its own RX directly. Everything the board transmits on UART2 it also receives on UART2, so no second device is required. UART0 (the USB cable) stays reserved for the console log, exactly as in Practice 3. If the three LEDs from Practice 3 are still wired to GPIO 25/26/27, they keep working unchanged.

4 Theory & Block Reference

No new SCADE modeling blocks appear in this practice: the model still produces plain boolean and integer outputs, exactly as generated in Practice 3. What is new lives entirely in the glue layer — the ESP-IDF UART driver and the frame that carries the model state. The reference below therefore documents driver APIs rather than SCADE blocks, and the UART driver is given in full because it is the one built end to end; SPI, I2C, and CAN are summarized so their trade-offs are understood before they appear in later work.

4.1. Why the protocol belongs in the glue, not the model

A synchronous dataflow model has no concept of a byte, a baud rate, or a wire — it computes one cycle of logic and hands its outputs to the caller. Serialization is an I/O concern of the execution platform, so it belongs in the same auditable glue layer that already owns GPIO and timing. Keeping it there means the model stays portable and certifiable: the same **TrafficLight** could be framed over UART today and CAN tomorrow with no change to a single generated line.

Block: UART driver (esp_driver_uart)

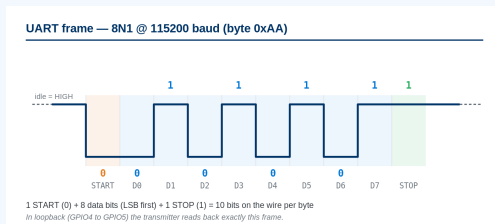


Figure 1: UART2 configured for 8N1 at 115200 baud with a loopback jumper (GPIO 4 to GPIO 5) — annotated view.

Key elements:

- **uart_config_t:** baud rate, data bits, parity, stop bits. 8N1 = 8 data bits, No parity, 1 stop bit — 10 bits on the wire per useful byte.
- **uart_driver_install:** allocates the RX (and optional TX) ring buffers.
- **uart_set_pin:** binds TX/RX to physical GPIOs (4/5 here).
- **uart_write_bytes** / **uart_read_bytes:** transmit and receive raw bytes; the read takes a timeout.

Behavior:

UART is asynchronous (no shared clock) and point-to-point: both ends must agree on the baud rate beforehand. A mismatched baud rate produces framing errors and “garbage” bytes, not a clean failure. At 115200 8N1 the effective throughput is $115200/10 = 11,520$ bytes/s.

Block: SPI, I2C, CAN - when UART is not enough

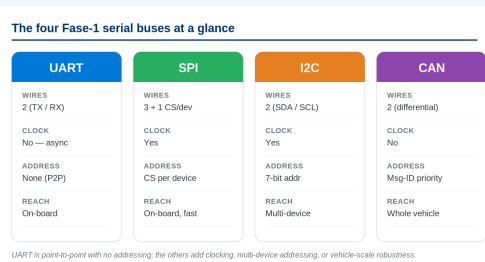


Figure 2: The four FASE-1 buses positioned by wires, addressing, and reach — annotated view.

Contrast:

- **SPI** (`esp_driver_spi`): synchronous, full-duplex, one clock plus MOSI/MISO/CS. Fastest, but one CS line per device; on-board, short range.
- **I2C** (`esp_driver_i2c`): synchronous, two wires (SDA/SCL) shared by many devices, each with a 7-bit address; needs pull-ups.
- **CAN/TWAI** (`esp_driver_twai`): differential two-wire bus across a whole vehicle; message-ID priority with non-destructive bit arbitration; needs a transceiver.

Behavior:

UART is point-to-point with no addressing; the other three add clocking, multi-device addressing, or vehicle-scale robustness. Choosing among them is a systems decision — reach, device count, and noise environment — not a coding one.

4.2. The telemetry frame designed for this practice

Raw bytes on a wire have no boundaries: a receiver that joins mid-stream cannot tell where a value starts. A *frame* solves this with a fixed structure. The frame used here is four bytes:

Byte	Name	Meaning
0	HEADER	Constant 0xAA — marks the start of a frame
1	TICK	Low byte of the model tick counter (0–119)
2	STATE	Bit0 = green, Bit1 = yellow, Bit2 = red
3	CHK	XOR of bytes 0–2 — a one-byte checksum

The header lets the receiver resynchronize; the checksum lets it reject a corrupted frame. Both are protocol concerns handled entirely in the glue.

5 Worked Example

5.1. Objective

Before framing anything, prove the UART loopback works with the simplest possible traffic: transmit a single byte on UART2 and read the same byte back. This isolates driver configuration and the loopback wire from the framing logic, and takes about 20 minutes.

5.2. Step-by-Step

1. **Create the project from Practice 3.**

Reuse the `lg_bridge` project so the `scade_gen` component and `swan_config.h` come along for free:

```
cd C:\esp
xcopy /e /i lg_bridge lg_telemetry
cd lg_telemetry
idf.py fullclean
```

2. **Wire the loopback.**

With the board unpowered, connect a single jumper from **GPIO 4** to **GPIO 5**. This ties UART2 TX directly to UART2 RX.

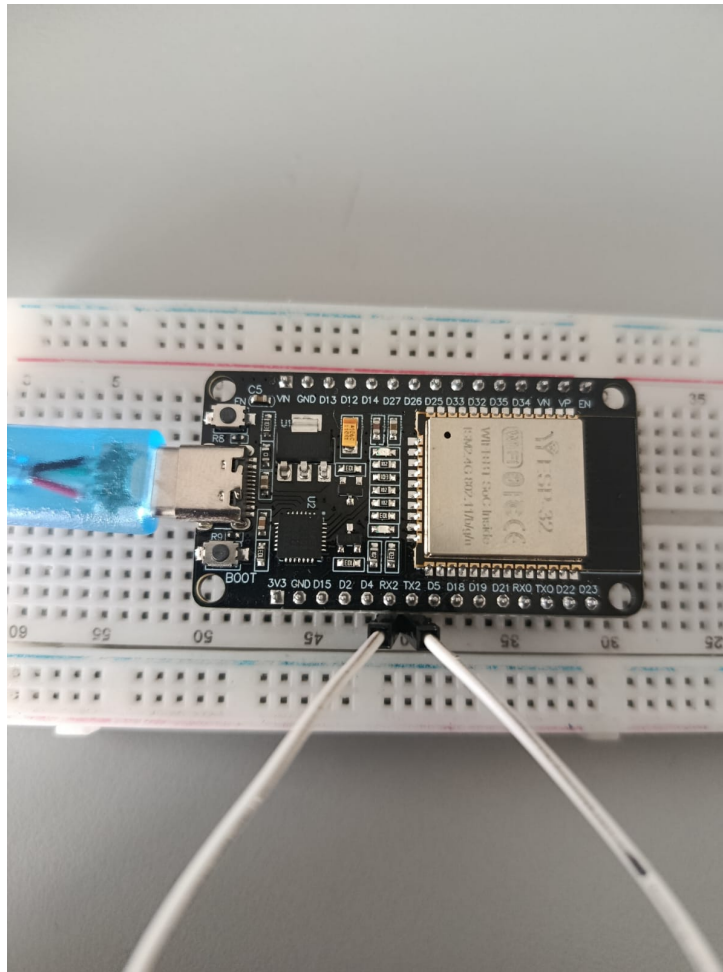


Figure 3: The single loopback jumper: GPIO 4 (TX) to GPIO 5 (RX).

3. Replace `main.c` with the loopback test.

Open notepad `main\main.c` and paste:

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/uart.h"
#include "esp_log.h"

#define U1_PORT UART_NUM_2
#define U1_TX    4
#define U1_RX    5
static const char *TAG = "UART";

void app_main(void)
{
    uart_config_t cfg = {
        .baud_rate = 115200,
        .data_bits = UART_DATA_8_BITS,
        .parity     = UART_PARITY_DISABLE,
        .stop_bits  = UART_STOP_BITS_1,
        .flow_ctrl  = UART_HW_FLOWCTRL_DISABLE,
        .source_clk = UART_SCLK_DEFAULT,
    };
    uart_driver_install(U1_PORT, 256, 0, 0, NULL, 0);
    uart_param_config(U1_PORT, &cfg);
    uart_set_pin(U1_PORT, U1_TX, U1_RX,
                UART_PIN_NO_CHANGE, UART_PIN_NO_CHANGE);

    while (1) {
        uint8_t tx = 0x41;           // 'A'
```

```

uint8_t rx = 0;
uart_write_bytes(U1_PORT, &tx, 1);
int n = uart_read_bytes(U1_PORT, &rx, 1, pdMS_TO_TICKS(50));
ESP_LOGI(TAG, "sent=0x%02X got=0x%02X (%s)",
          tx, rx, (n == 1 && rx == tx) ? "OK" : "FAIL");
vTaskDelay(pdMS_TO_TICKS(500));
}
}

```

4. Declare the UART dependency.

Edit main\CMakeLists.txt:

```

idf_component_register(SRCS "main.c"
                      INCLUDE_DIRS "."
                      REQUIRES scade_gen esp_driver_uart)

```

5. Build, flash, and verify.

Run `idf.py -p COM8 flash monitor`. Every 500 ms the console must print `sent=0x41 got=0x41 (OK)`. Pull the jumper for one second and the line changes to `got=0x00 (FAIL)` — direct proof that the byte really traveled through the wire, not through software.

```

I (46981) UART: sent=0x41 got=0x41 (OK)
I (47481) UART: sent=0x41 got=0x41 (OK)
I (47981) UART: sent=0x41 got=0x41 (OK)
I (48481) UART: sent=0x41 got=0x41 (OK)
I (48981) UART: sent=0x41 got=0x41 (OK)
I (49481) UART: sent=0x41 got=0x41 (OK)
I (49981) UART: sent=0x41 got=0x41 (OK)
I (50481) UART: sent=0x41 got=0x41 (OK)
I (50981) UART: sent=0x41 got=0x41 (OK)
I (51481) UART: sent=0x41 got=0x41 (OK)
I (51981) UART: sent=0x41 got=0x41 (OK)
I (52481) UART: sent=0x41 got=0x41 (OK)
I (52981) UART: sent=0x41 got=0x41 (OK)
I (53481) UART: sent=0x41 got=0x41 (OK)
I (53981) UART: sent=0x41 got=0x41 (OK)
I (54481) UART: sent=0x41 got=0x41 (OK)
I (54981) UART: sent=0x41 got=0x41 (OK)
I (55481) UART: sent=0x41 got=0x41 (OK)
I (55981) UART: sent=0x41 got=0x41 (OK)
I (56481) UART: sent=0x41 got=0x41 (OK)
I (57031) UART: sent=0x41 got=0x00 (FAIL)
I (57581) UART: sent=0x41 got=0x00 (FAIL)
I (58131) UART: sent=0x41 got=0x00 (FAIL)
I (58681) UART: sent=0x41 got=0x00 (FAIL)
I (59231) UART: sent=0x41 got=0x00 (FAIL)
I (59781) UART: sent=0x41 got=0x00 (FAIL)
I (60331) UART: sent=0x41 got=0x00 (FAIL)
I (60881) UART: sent=0x41 got=0x00 (FAIL)
I (61431) UART: sent=0x41 got=0x00 (FAIL)
I (61981) UART: sent=0x41 got=0x00 (FAIL)
I (62531) UART: sent=0x41 got=0x00 (FAIL)
I (63081) UART: sent=0x41 got=0x00 (FAIL)

```

Figure 4: Loopback confirmed: transmitted and received bytes match.

6 Main Activity

6.1. Description

Extend the loopback into a real telemetry link. Each 100 ms cycle, step the unchanged `TrafficLight` model, pack its state into the four-byte frame defined in Section 4, transmit it on UART2, read it back through the loopback, validate the header and checksum, and log the result on the console. The model stays byte-agnostic; all framing, transmission, and validation live in the glue layer.

6.2. Functional Specifications

- **Spec 1:** The generated `scade_gen` component is reused from Practice 3 with zero changes; the model is not regenerated.
- **Spec 2:** Each cycle builds the frame `[0xAA][tick][state][chk]`, where `state` bit0/1/2 = green/yellow/red and `chk` is the XOR of the first three bytes.
- **Spec 3:** The frame is transmitted on UART2 (GPIO 4) and read back on UART2 (GPIO 5) through the loopback jumper within the same cycle.
- **Spec 4:** The receiver validates `HEADER == 0xAA` and `chk` before trusting the frame; a failed check is logged as `RX-BAD` and does not update the reported state.
- **Spec 5:** The console prints one line per cycle showing the transmitted bytes and the decoded, validated state, matching the LED behaviour and the Practice 3 simulation.

6.3. Activity Steps

1. Confirm the generated interface.

Open `components\scade_gen\operator1_module0.h` and re-confirm the step signature and context type from Practice 3: `operator1_module0(&cont, &green, &yellow, &red, &ctx)` with context `outC_operator1_module0`. Adapt the names below if your generated header differs.

2. Write the telemetry glue.

Open notepad `main\main.c` and paste the full program:

```
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/uart.h"
#include "driver/gpio.h"
#include "esp_log.h"
#include "operator1_module0.h" // generated (TrafficLight)

#define U1_PORT UART_NUM_2
#define U1_TX 4
#define U1_RX 5
#define HDR 0xAA
#define LED_RED 25
#define LED_YEL 26
#define LED_GRN 27
static const char *TAG = "TLM";

static outC_operator1_module0 ctx;
static swan_int32 cont;
```

```

static swan_bool green, yellow, red;

void app_main(void)
{
    // UART2 as the telemetry link (8N1, 115200)
    uart_config_t cfg = {
        .baud_rate = 115200, .data_bits = UART_DATA_8_BITS,
        .parity = UART_PARITY_DISABLE, .stop_bits = UART_STOP_BITS_1,
        .flow_ctrl = UART_HW_FLOWCTRL_DISABLE,
        .source_clk = UART_SCLK_DEFAULT,
    };
    uart_driver_install(U1_PORT, 256, 0, 0, NULL, 0);
    uart_param_config(U1_PORT, &cfg);
    uart_set_pin(U1_PORT, U1_TX, U1_RX,
        UART_PIN_NO_CHANGE, UART_PIN_NO_CHANGE);

    gpio_set_direction(LED_RED, GPIO_MODE_OUTPUT);
    gpio_set_direction(LED_YEL, GPIO_MODE_OUTPUT);
    gpio_set_direction(LED_GRN, GPIO_MODE_OUTPUT);

    operator1_reset_module0(&ctx);
    while (1) {
        operator1_module0(&cont, &green, &yellow, &red, &ctx);
        gpio_set_level(LED_RED, red);
        gpio_set_level(LED_YEL, yellow);
        gpio_set_level(LED_GRN, green);

        // ---- build the frame (glue layer, not the model) ----
        uint8_t state = (green ? 1 : 0) | (yellow ? 2 : 0) | (red ? 4 : 0);
        uint8_t tx[4];
        tx[0] = HDR;
        tx[1] = (uint8_t)cont;
        tx[2] = state;
        tx[3] = tx[0] ^ tx[1] ^ tx[2]; // checksum

        // ---- transmit, then read back through the loopback ----
        uart_write_bytes(U1_PORT, tx, 4);
        uint8_t rx[4] = {0};
        int n = uart_read_bytes(U1_PORT, rx, 4, pdMS_TO_TICKS(20));

        // ---- validate header + checksum ----
        uint8_t chk = rx[0] ^ rx[1] ^ rx[2];
        if (n == 4 && rx[0] == HDR && rx[3] == chk) {
            ESP_LOGI(TAG, "TX %02X %02X %02X %02X | RX tick=%d G=%d Y=%d R=%d",
                tx[0], tx[1], tx[2], tx[3], rx[1],
                rx[2] & 1, (rx[2] >> 1) & 1, (rx[2] >> 2) & 1);
        } else {
            ESP_LOGW(TAG, "RX-BAD n=%d hdr=%02X chk=%02X/%02X", n, rx[0], rx[3], chk);
        }
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}

```

3. Declare the dependencies.

Edit `main\CMakeLists.txt`. Because the glue now uses UART and GPIO, both driver components must be listed:

```

idf_component_register(SRCS "main.c"
    INCLUDE_DIRS "."
    REQUIRES scade_gen esp_driver_uart esp_driver_gpio)

```

4. Build, flash, and verify against the model.

`idf.py -p COM8 flash monitor`. Confirm the decoded RX state advances $G \rightarrow Y \rightarrow R$ on the same 5/2/5s schedule as Practice 3, that the TX checksum byte equals the XOR of the first three, and that pulling the loopback jumper immediately produces

RX-BAD lines.

```

ESP-IDF 5.5 CMD - "C:\Espress
+  v
I (208) efuse_init: Min chip rev:    v0.0
I (212) efuse_init: Max chip rev:    v3.99
I (216) efuse_init: Chip rev:        v3.1
I (220) heap_init: Initializing. RAM available for dynamic allocation:
I (226) heap_init: At 3FFAE6E0 len 00001920 (6 KiB): DRAM
I (231) heap_init: At 3FFB3110 len 0002CEF0 (179 KiB): DRAM
I (237) heap_init: At 3FFE0440 len 00003AE0 (14 KiB): D/IRAM
I (242) heap_init: At 3FFE4350 len 0001BCB0 (111 KiB): D/IRAM
I (248) heap_init: At 4008DEB4 len 0001214C (72 KiB): IRAM
I (254) spi_flash: detected chip: generic
I (257) spi_flash: flash io: dio
W (259) spi_flash: Detected size(4096k) larger than the size in the binary image header
I (272) main_task: Started on CPU0
I (282) main_task: Calling app_main()
I (282) TLM: TX AA 00 01 AB | RX tick=0 G=1 Y=0 R=0
I (382) TLM: TX AA 01 01 AA | RX tick=1 G=1 Y=0 R=0
I (482) TLM: TX AA 02 01 A9 | RX tick=2 G=1 Y=0 R=0
I (582) TLM: TX AA 03 01 A8 | RX tick=3 G=1 Y=0 R=0
I (682) TLM: TX AA 04 01 AF | RX tick=4 G=1 Y=0 R=0
I (782) TLM: TX AA 05 01 AE | RX tick=5 G=1 Y=0 R=0
I (882) TLM: TX AA 06 01 AD | RX tick=6 G=1 Y=0 R=0
I (982) TLM: TX AA 07 01 AC | RX tick=7 G=1 Y=0 R=0
I (1082) TLM: TX AA 08 01 A3 | RX tick=8 G=1 Y=0 R=0
I (1182) TLM: TX AA 09 01 A2 | RX tick=9 G=1 Y=0 R=0
I (1282) TLM: TX AA 0A 01 A1 | RX tick=10 G=1 Y=0 R=0
I (1382) TLM: TX AA 0B 01 A0 | RX tick=11 G=1 Y=0 R=0
I (1482) TLM: TX AA 0C 01 A7 | RX tick=12 G=1 Y=0 R=0
I (1582) TLM: TX AA 0D 01 A6 | RX tick=13 G=1 Y=0 R=0
I (1682) TLM: TX AA 0E 01 A5 | RX tick=14 G=1 Y=0 R=0
I (1782) TLM: TX AA 0F 01 A4 | RX tick=15 G=1 Y=0 R=0
I (1882) TLM: TX AA 10 01 BB | RX tick=16 G=1 Y=0 R=0
I (1982) TLM: TX AA 11 01 BA | RX tick=17 G=1 Y=0 R=0
I (2082) TLM: TX AA 12 01 B9 | RX tick=18 G=1 Y=0 R=0
I (2182) TLM: TX AA 13 01 B8 | RX tick=19 G=1 Y=0 R=0
I (2282) TLM: TX AA 14 01 BF | RX tick=20 G=1 Y=0 R=0
I (2382) TLM: TX AA 15 01 BE | RX tick=21 G=1 Y=0 R=0
I (2482) TLM: TX AA 16 01 BD | RX tick=22 G=1 Y=0 R=0
I (2582) TLM: TX AA 17 01 BC | RX tick=23 G=1 Y=0 R=0
I (2682) TLM: TX AA 18 01 B3 | RX tick=24 G=1 Y=0 R=0
I (2782) TLM: TX AA 19 01 B2 | RX tick=25 G=1 Y=0 R=0
I (2882) TLM: TX AA 1A 01 B1 | RX tick=26 G=1 Y=0 R=0

```

Figure 5: Console showing framed telemetry sent and validated on loopback, cycle by cycle.

7 Expected Results

Expected Results

- **SCADE Simulation:** Unchanged from Practice 3 — `cont` ramps 0–119 and wraps, with exactly one of `green/yellow/red` true per cycle. The model is not modified in this practice.
- **Console / UART Output:** One line per 100ms cycle, showing the four transmitted bytes and the decoded, checksum-validated state read back on the loopback:


```

I (312) TLM: TX AA 00 01 AB | RX tick=0 G=1 Y=0 R=0
I (5312) TLM: TX AA 32 02 98 | RX tick=50 G=0 Y=1 R=0
I (7312) TLM: TX AA 46 04 EC | RX tick=70 G=0 Y=0 R=1
W (9100) TLM: RX-BAD n=0 hdr=00 chk=00/00 (jumper removed)

```
- **Physical Behavior:** With the loopback jumper in place, every cycle validates and the optional LEDs cycle green (5s) → yellow (2s) → red (5s). Removing the jumper stops reception — RX-BAD appears — while the LEDs keep cycling,

proving the model runs independently of the link.

8 Assessment Questionnaire

Answer each question based on your specific implementation. Justify your responses with references to your design decisions.

Q1. [*Comprehension*] In your frame, byte 0 is a constant `0xAA` and byte 3 is the XOR of the first three. Explain the distinct job each one does for the receiver, and why the header alone is not enough to trust a frame nor the checksum alone enough to locate one.

Q2. [*Analysis*] UART is asynchronous: no clock travels with the data. Predict what the receiver would read if UART2 were configured at 115200 baud on transmit but 57600 baud on receive, and explain — in terms of when the receiver samples each bit — why the result is corrupted bytes rather than a clean error.

Q3. [*Analysis*] Your telemetry frame is four fixed bytes. Compare this fixed-length binary frame with an ASCII line such as `"tick=50,G=0,Y=1,R=0\n"` for the same data, with respect to bandwidth at 115200 baud, ease of parsing, and robustness to a mid-stream join. Argue which is better for a flight-data link.

Q4. [*Synthesis*] A reviewer asks you to also transmit a rolling frame counter so dropped frames can be detected. Extend the frame and the glue to carry an 8-bit sequence number, state what the receiver must now check, and explain why none of this touches the generated `scade_gen` code or `swan_config.h`.

Q5. [*Critical*] The one-byte XOR checksum detects any single-bit error but misses some multi-bit errors. Identify one concrete corruption your checksum would fail to catch, name a stronger mechanism used by CAN for the same job, and argue whether an XOR checksum is acceptable for a landing-gear status link.

9 Conclusion

Write a concise conclusion reflecting on what you implemented, what you observed, and what challenges you encountered. Connect your findings to model-based design for critical systems — in particular, what it means that a communication protocol could be added, tested, and validated without changing or even regenerating the model that produces the data.

10 Bonus Challenge

★ Bonus Challenge

Challenge:

Turn the loopback into a resynchronizing receiver. Instead of reading exactly four bytes each cycle, read a byte at a time and implement a small state machine in the glue that scans for the `0xAA` header, then collects the next three bytes and validates the checksum — so that even if the receiver starts mid-frame it recovers on the next header. Inject a deliberate desync (send one extra byte occasionally) and show that the receiver re-locks. Keep every change in the glue layer.

Hint:

You already know the header is unique-ish (`0xAA`); the trick is to never advance past a byte that is not the header while searching. Think of a two-state machine: `SEEK_HEADER` and `COLLECT`. The checksum is what tells you a collected frame is real before you trust it.

11 Troubleshooting

This section is populated with real issues encountered during development. If you run into a problem not listed here, document it using the same format below.

Issue: Every read returns RX-BAD with n=0

Symptom: No bytes are ever received; `uart_read_bytes` returns 0 on every cycle.

Cause: Usually the loopback jumper is missing, loose, or on the wrong pins; occasionally TX/RX are swapped in `uart_set_pin`.

Fix: Confirm a solid jumper from GPIO 4 to GPIO 5 and that `uart_set_pin` lists TX (4) before RX (5). The most common cause is a pin mismatch: the firmware and the jumper must point at the same two GPIOs. The Worked Example single-byte test isolates this before the full frame.

Issue: Compile fails: “driver/uart.h: No such file” or missing UART symbols

Symptom: `main.c` fails to compile or link on the UART calls.

Cause: `esp_driver_uart` is not in the `REQUIRES` list of `main/CMakeLists.txt`.

Fix: Add it: `REQUIRES scade_gen esp_driver_uart esp_driver_gpio`. This is the same class of error as the `esp_driver_gpio` one from Practice 3.

Issue: Bytes arrive but the checksum never matches

Symptom: `n=4` and the header is correct, but frames are always logged as `RX-BAD`.

Cause: The checksum is computed over a different set of bytes on transmit and receive (e.g. including `tx[3]` in the XOR), or the read timeout is too short and returns a partial frame that shifts the bytes.

Fix: XOR exactly bytes 0–2 on both sides, and give `uart_read_bytes` enough timeout (20 ms is ample for four bytes at 115200).

Issue: Console shows garbage instead of the log

Symptom: The monitor prints unreadable characters.

Cause: The monitor baud rate does not match the console; or UART2 was accidentally mapped onto the UART0 console pins (GPIO 1/3).

Fix: Keep UART2 on GPIO 4/5, never on 1/3, and monitor at 115200. UART0 (USB) must stay dedicated to the console.